Machine Translated by Google

PISCINE OBJECT

Module 02 - UML

Résumé : Ce document vous présentera les bases du langage de modélisation universel.

Version : 1.00

Machine Translated by Google

Contenu

je Préambule 2

II Principe 3

III Partie obligatoire 5

Bonus IV 7

V. Soumission et évaluations par les pairs 9

Machine Translated by Google

Chapitre I

Préambule

Aux débuts de la programmation, il n'y avait pas vraiment de « règles » concernant ce qui était propre ou non. C'était un peu le Far West !

Depuis, les ingénieurs ont commencé à réfléchir à un moyen de se coordonner pour produire un code plus facile à maintenir et à lire.

Pour ce faire, ils ont créé leur propre langage, appelé Unified Modeling Language (UML).
Ce langage leur servira, à partir de maintenant, à transformer une architecture logicielle en un graphe, permettant de discuter, d'échanger et de comprendre beaucoup plus facilement ce qui doit être fait !

Machine Translated by Google

Chapitre II

Principe

UML est un langage de modélisation graphique permettant la représentation visuelle de différents aspects d'un système logiciel. Les diagrammes UML sont des représentations graphiques de ces aspects, et il existe plusieurs types de diagrammes UML utilisables selon les besoins de modélisation.

Il existe plusieurs types de diagrammes, tels que :

- Diagramme de classes
- Diagramme de séquence
- Diagramme d'activité
- Diagramme de transition d'état
- Diagramme de cas d'utilisation
- Schéma de déploiement
- Etc

Chaque type de diagramme UML possède ses propres caractéristiques et sert à représenter différents aspects d'un système. La combinaison de ces diagrammes permet de créer une représentation complète et détaillée d'un système logiciel.

Dans ce module, vous découvrirez le diagramme de classes, l'un des plus utilisés en UML. Il permet de représenter les classes, les attributs, les méthodes et les relations entre ces éléments.

Machine Translated by Google

Les classes sont représentées par des rectangles contenant le nom de la classe, ses attributs et méthodes.
Les attributs sont indiqués sous le nom de la classe et les méthodes sous les attributs. Les relations entre les classes sont représentées par des flèches.

Il existe plusieurs types de flèches qui représentent différentes relations entre les classes, par exemple, de manière non exhaustive :

- Flèche d'héritage, représentée par une ligne continue avec une flèche pointant vers le parent classe.
- Flèche d'implémentation, représentée par une ligne pointillée avec une flèche pointant vers l'interface. Cette flèche indique que la classe implémente les méthodes définies dans l'interface.
- Flèche d'agrégation, représentée par un losange ouvert avec une flèche pointant vers la classe contenant.
- Flèche de composition, représentée par un losange plein avec une flèche pointant vers la classe contenant.

Enfin, il est important de noter que les relations dans un diagramme de classes sont généralement « descendantes ».
Cela signifie que la classe parente doit se trouver en haut du diagramme, et les classes enfants en dessous, la flèche pointant vers le haut.
Cette convention de flèches est utilisée pour faciliter la lecture et la compréhension du diagramme.

Machine Translated by Google

Chapitre III

Partie obligatoire

	Exercice 00
Exercice 00: Car composition	
Répertoire de dépôt : ex00/	
Fichiers à remettre : subject.png	
Fonctions autorisées : Aucune	

Pour cet exercice, vous devez créer un diagramme, sous forme de fichier png, nommé subject.png.

Cette image doit représenter graphiquement l'ensemble des objets suivants :

- La structure LinkablePart est une classe virtuelle, nécessitant l'implémentation d'une méthode void execute(float p_pression) pour être instanciée.
- La structure Wheel est une classe contenant une méthode void executeRotation(float p_force); qui permet à une roue d'effectuer une rotation avec une force donnée.
- La structure Gear est une classe contenant un démultiplicateur de variable entière qui stocke le démultiplicateur d'un engrenage.

La structure GearLever est une classe héritant de Singleton<GearLever> et contenant un tableau d'objets Gear et une variable entière level. Elle possède une méthode void change() permettant de changer le rapport engagé et une méthode Gear* activeGear() renvoyant un pointeur vers le rapport engagé.

- La structure Transmission est une classe contenant un vecteur de pointeurs vers des objets Wheel et une méthode void activate(float p_force); qui active la transmission avec une force donnée.
- La structure Crankshaft est une classe contenant un pointeur vers un objet Transmission et une méthode void receiveForce(float p_volume); qui reçoit une force dans un volume donné.
- La structure ExplosionChamber est une classe contenant un pointeur vers un objet Crankshaft et une méthode void fill(float p_volume); qui remplit la chambre avec un volume donné.

Machine Translated by Google

La structure Injector est une classe héritant de LinkablePart, contenant un pointeur vers un objet ExplosionChamber et une méthode void execute(float p_pression); qui exécute l'injection à une pression donnée.

La structure Motor est une classe contenant des objets de type Injector, ExplosionChamber et Crankshaft. Elle possède une méthode void connectToTransmission(Transmission* p_transmission); qui connecte le moteur à une transmission donnée.

- La structure Pedal est une classe contenant un pointeur vers un objet LinkablePart et deux méthodes : void setTarget(LinkablePart* p_part) ; qui définit la cible de la pédale sur une partie donnée, et void use(float p_pression) ; qui utilise la pédale avec une pression donnée.

- La structure Direction est une classe contenant un tableau d'objets Wheel et une méthode void turn(float p_angle); qui fait tourner la direction d'un angle donné.

La structure DAE est une classe contenant un pointeur vers un objet Direction et une variable flottante force. Elle possède une méthode void use(float p_angle); qui utilise la structure DAE avec un angle donné.

- La structure SteerWheel est une classe contenant un pointeur vers un objet DAE et méthode(s) void turn(float p_angle) qui prend un float comme entrée.

- La structure Brake est une classe contenant un pointeur vers un objet Wheel et une ou plusieurs méthodes void execute(float p_force) et void attackWheel(Wheel* p_wheel) qui prennent respectivement un float et un pointeur vers un objet Wheel comme entrées.

- La structure BrakeController est une classe qui hérite de LinkablePart et contient un tableau d'objets Brake et une ou plusieurs méthodes void execute(float p_pression) qui prennent un float en entrée.

- La structure Cockpit est une classe contenant des objets Pedal, SteerWheel et GearLever et les méthodes qui gèrent ces objets.

- La structure Electronics est une classe contenant un objet DAE.

- La structure Voiture est une classe contenant des objets Contrôleur de frein, Direction, Transmission, Moteur, Électronique et Cockpit.

Machine Translated by Google

Chapitre IV

Bonus

	Exercice 00
Bonus 00 : Améliorations des diagrammes	
Répertoire de dépôt : ex00/	
Fichiers à remettre : subject.png	
Fonctions autorisées : Aucune	

Plusieurs bonus sont proposés pour approfondir la modélisation du système.
Il est tout d'abord proposé de modifier l'architecture du système afin de corriger les liens ascendants présents dans l'exemple. En effet, UML recommande généralement d'établir uniquement des relations descendantes ou horizontales entre les différentes classes d'un système. La modification de l'architecture permet de garantir que les relations entre les classes sont conformes aux recommandations UML.

Il est ensuite proposé de transformer les structures du système en classes et de répartir leurs attributs et méthodes entre publics et privés. Cette transformation permet une meilleure modélisation des différentes entités du système et rend le code plus facilement compréhensible pour les développeurs.

Il est également proposé de travailler sur la représentation graphique du système, en veillant à ce que les différentes unités soient réparties en blocs distincts pour une meilleure lisibilité. Il est également recommandé de représenter les définitions d'association (1..*, etc.) lorsque cela s'avère nécessaire, pour une modélisation plus précise.

Enfin, un ou plusieurs diagrammes de séquence pour la direction, le freinage et l'accélération peuvent être proposés. Ces diagrammes permettent de modéliser les interactions entre les différents éléments du système dans des scénarios spécifiques et contribuent à une meilleure compréhension de son fonctionnement dans ces situations.

Une fois que vous aurez terminé l'attribution de ces bonus, vous serez libre d'ajouter tous les bonus que vous jugerez nécessaires.

Machine Translated by Google

La partie bonus ne sera évaluée que si la partie obligatoire est PARFAITE. Par « parfaite », on entend que la partie obligatoire a été intégralement réalisée et validée par l'évaluateur. Si vous n'avez pas satisfait à TOUTES les exigences obligatoires, votre partie bonus ne sera pas évaluée.

Machine Translated by Google

Chapitre V

Soumission et évaluations par les pairs

Pour cette évaluation, vous devez fournir tous les documents que vous jugez nécessaires à la compréhension de votre diagramme.
Vous ne serez évalué que sur ce qui se trouve dans le dépôt.
L'évaluation portera principalement sur l'explication de ce que vous avez fait dans votre diagramme, alors préparez-vous à discuter de ce sujet !

Veuillez déposer votre travail dans votre dépôt Git comme d'habitude. Seul le contenu de votre dépôt sera évalué lors de la soutenance. N'hésitez pas à revérifier les noms de vos dossiers et fichiers pour vous assurer de leur exactitude.